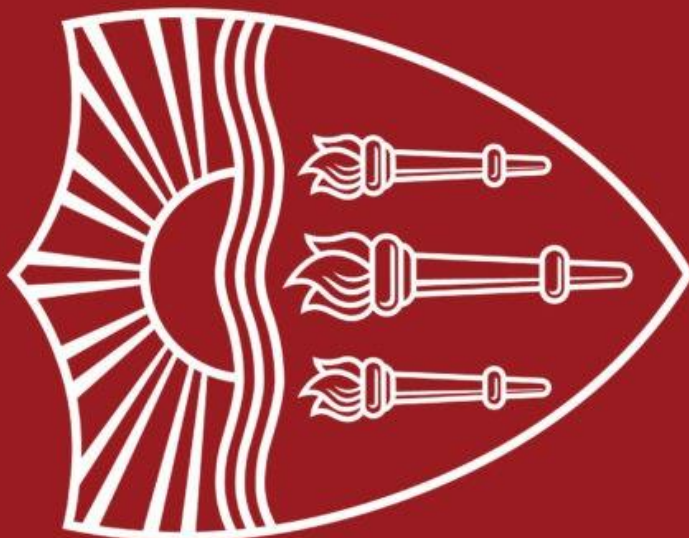




Lecture 7:

Dense Word Embedding & Feed-forward Neural Nets

Instructor: Xiang Ren
USC CSCI 544 Applied NLP
Fall 2026

Lecture Outline

- Announcements
- Recap: Sparse word embeddings
- Dense Word Embeddings, word2vec
- GloVe
- Word Embeddings: Properties and Evaluation
- Feed-forward Neural Nets

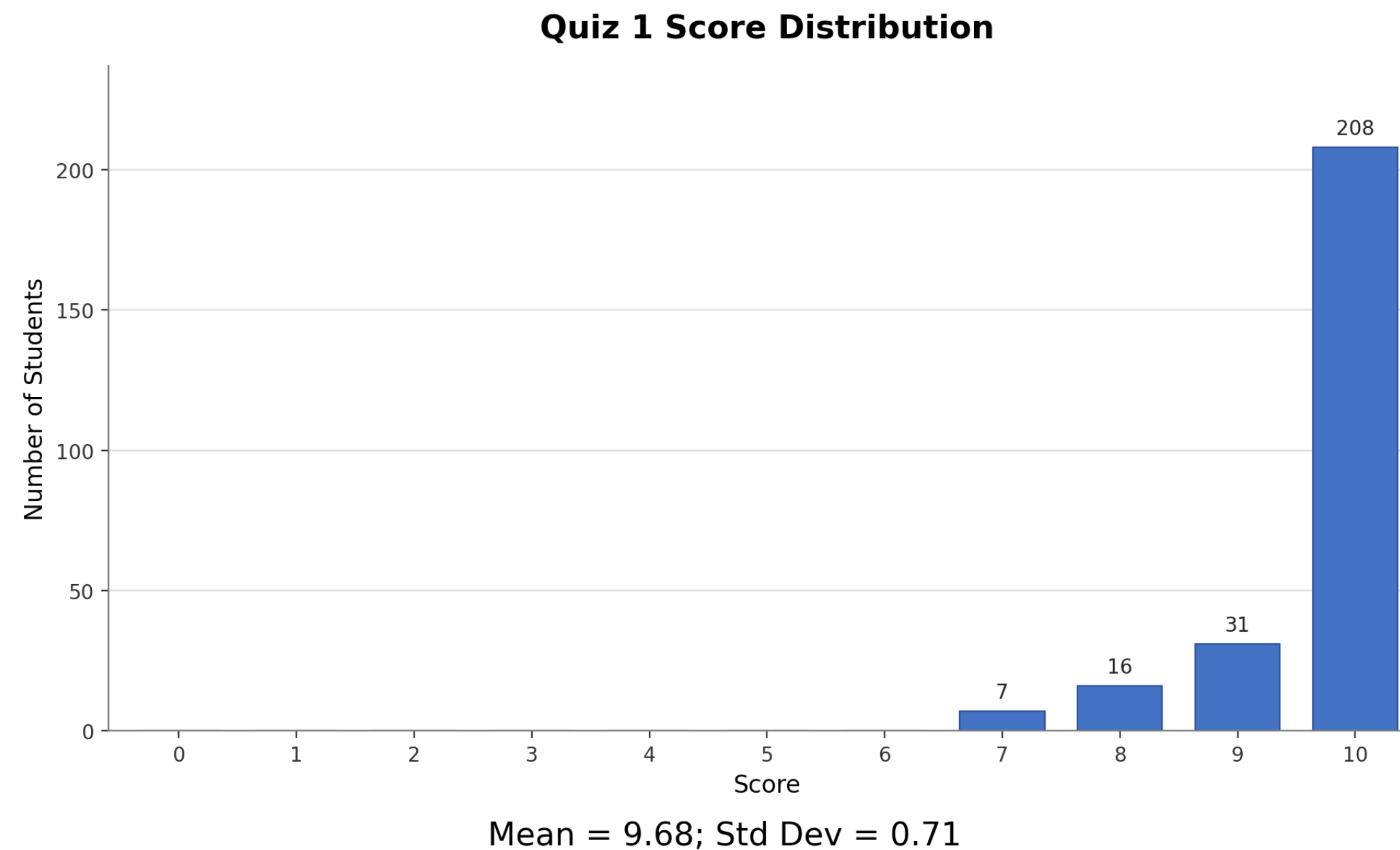
Announcements + Logistics

- 46 Teams in the end
 - 46 full groups (project 1-47, no project 41)
 - 46 project proposals submitted (will be graded in ~1 week)
 - looking for 2 project groups to add a 7th member (email hjarai@usc.edu)
- Quiz logistics for DEN students
 - 72-hour window to start the Quiz (10 mins)
 - Use Rsoondus lockdown browser
 - Quiz published post class

Announcements + Logistics

Quiz 1

- Including excused and DEN students, total of 254 students attended; and 23 did not.



Announcements + Logistics

- HW1 release – Sep 15
 - TA will make an announcement on Brightspace
 - Submission via Brightspace

HW1: Sentiment analysis of product reviews

Amazon Office Products reviews 100 points Individual assignment

50 pts Prepare reviews and extract features

Clean text, remove stop words, lemmatize, and build TF-IDF.
200,000 reviews with an 80/20 train/test split.

30 pts Train and evaluate three classifiers

Perceptron, Logistic Regression, and Multinomial Naive Bayes.
Report train/test accuracy, precision, recall, and F1.

20 pts Build a bigram language model

Handle unknown words and compare add-k smoothing.
Evaluate corpus perplexity for $k = 1, 0.1, \text{ and } 0.01$.

Due October 13, 2026 at 2:59 PM Pacific Time

HW1: Sentiment analysis of product reviews

Amazon Office Products reviews 100 points Individual assignment

Submission

sentiment.py

- Submit via Brightspace -> Activities -> Assignments -> HW1
- Python 3.12
- No written report required

Perceptron & Naive Bayes

- Background and references are in the assignment.
- Use the specified sklearn implementations.
- No algorithm derivations required.

Due October 13, 2026 at 2:59 PM Pacific Time

Recap: Sparse (Count-based) Word Embeddings

Word Meaning via Language Use

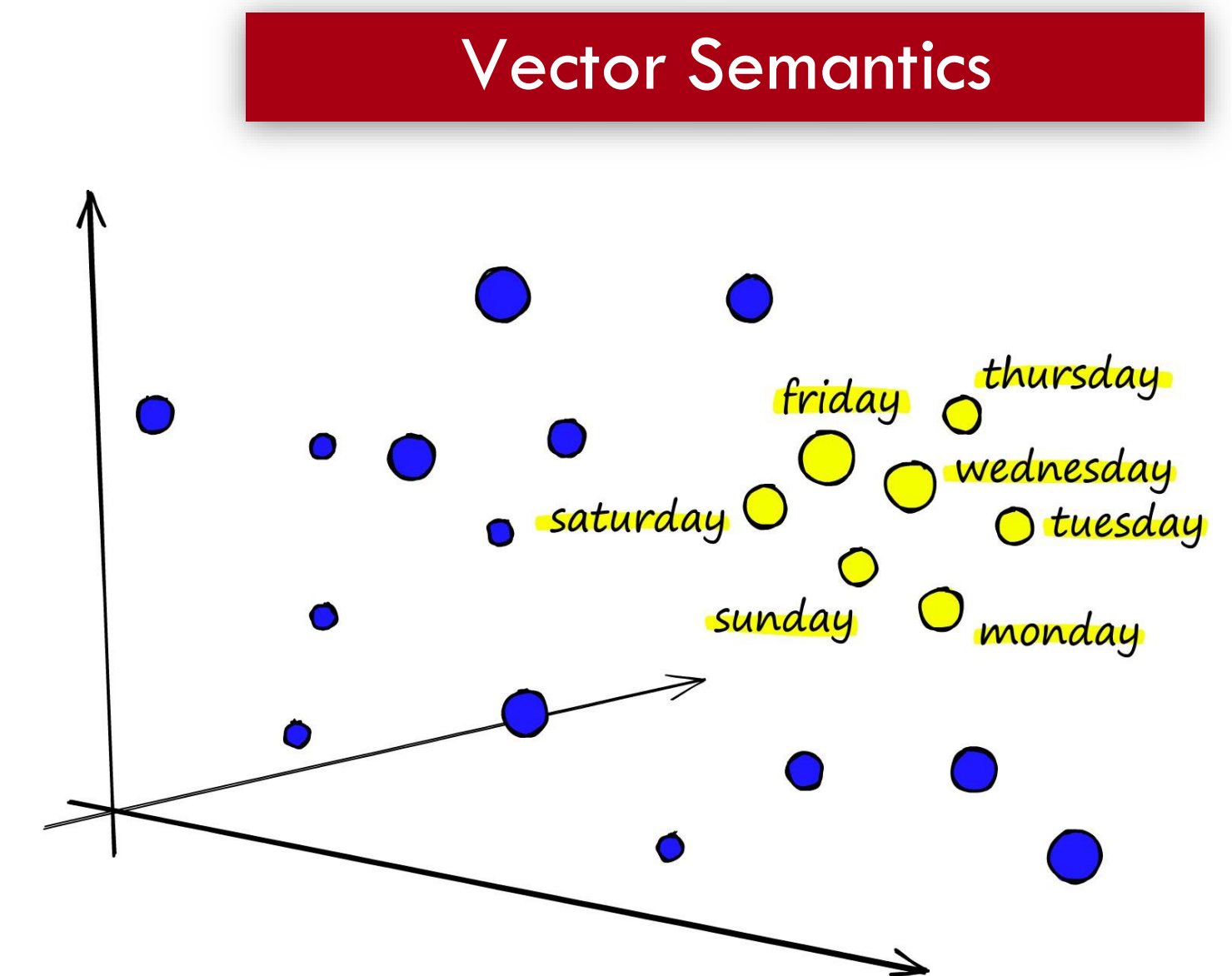
- The meaning of a word can be given by its distribution in language usage
 - Word “usage”: word environments / contexts
 - Neighboring words or grammatical environments
- Intuitions: Zellig Harris (1954):
 - “oculist and eye-doctor ... occur in almost the same environments”
 - “If A and B have almost identical environments we say that they are synonyms.”
- A bottle of tesgüino is on the table
- Everybody likes tesgüino
- Tesgüino makes you drunk
- We make tesgüino out of corn.



Two words are similar if they have similar word contexts

Word Embeddings

- Represent a word as a point in a multidimensional semantic space
 - Space itself constructed from distribution of word neighbors
- Called an "embedding" because it's embedded into a space
- Fine-grained model of meaning for similarity



Every modern NLP algorithm uses embeddings as the representation of word meaning

n-grams as One-hot Vectors

vocabulary

i

hate

love

the

movie

film

movie = $\langle 0,0,0,0,1,0 \rangle$

film = $\langle 0,0,0,0,0,1 \rangle$

Unigram Vectors: Represent each word as a vector of zeros with a single 1 identifying the index of the word

One hot vector

How can we compute a vector representation such that the dot product correlates with word similarity?

Dot product is zero! These vectors are orthogonal

Term-document matrix

Let us consider a collection of documents and count how frequently a word (term) appears in each. A document could be a play or a Wikipedia article. In general, documents can be anything; we often call each paragraph a document!

Each **document** is represented by a vector of words

	As You Like It	Twelfth Night	Julius Caesar	Henry V
battle	1	0	7	13
good	114	80	62	89
fool	36	58	1	4
wit	20	15	2	3

Word-word co-occurrence matrix

is traditionally followed by **cherry** pie, a traditional dessert
 often mixed, such as **strawberry** rhubarb pie. Apple pie
 computer peripherals and personal **digital** assistants. These devices usually
 a computer. This includes **information** available on the internet

	a	computer	pie
cherry	1	0	1
strawberry	0	0	2
digital	0	1	0
information	1	1	0

Pointwise Mutual Information (PMI)

- PMI between two words:
$$PMI(w_1, w_2) = \log \frac{P(w_1, w_2)}{P(W_1)P(w_2)}$$
 - (association) Do words w_1 and w_2 co-occur more than if they were independent?
- PMI ranges from $-\infty$ to $+\infty$
 - Negative values are problematic: words are co-occurring less than we expect by chance
 - Only reliable under an enormous corpora
 - Imagine w_1 and w_2 whose probability of occurrence is each 10^{-6}
 - Hard to be sure $P(w_1, w_2)$ is significantly different than 10^{-12}
 - So we just replace negative PMI values by 0

- Positive PMI

$$PPMI(w_1, w_2) = \max(0, \log \frac{P(w_1, w_2)}{P(W_1)P(w_2)})$$

Computing PPMI on a term-context matrix

Context c

Term w

	computer	data	result	pie	sugar	count(w)
cherry	2	8	9	442	25	486
strawberry	0	0	1	60	19	80
digital	1670	1683	85	5	4	3447
information	3325	3982	378	5	13	7703
count(context)	4997	5673	473	512	61	11716

Frequency $f(w_i, c_j)$ or f_{ij} is the # times w_i occurs in context c_j

$$P_{ij} = \frac{f_{ij}}{\sum_{t=1}^W \sum_{k=1}^C f_{tk}}$$

$$P_i = \sum_{j=1}^C P_{ij}$$

$$P_j = \sum_{i=1}^W P_{ij}$$

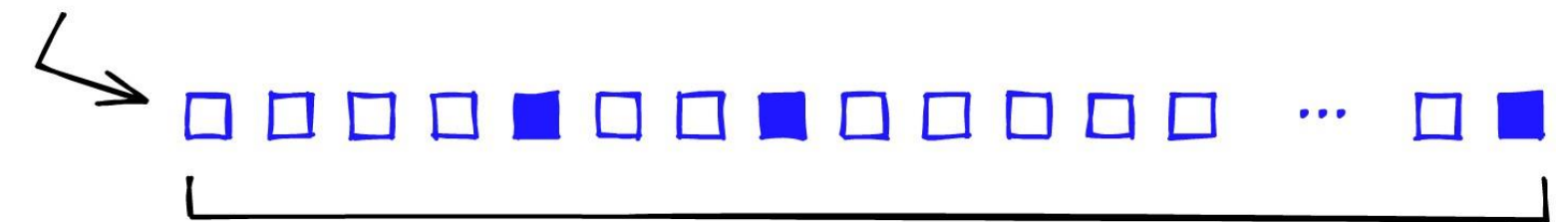
$$PPMI(w_i, c_j) = PPMI_{i,j} = \max(0, \log \frac{P_{ij}}{P_i P_j})$$

The problem...

- PMI vectors are
 - long (length $|V| = 20,000$ to $50,000$)
 - sparse (most elements are zero)
- Alternative: learn vectors which are
 - short (length 50-1000)
 - dense (most elements are non-zero)

sparse

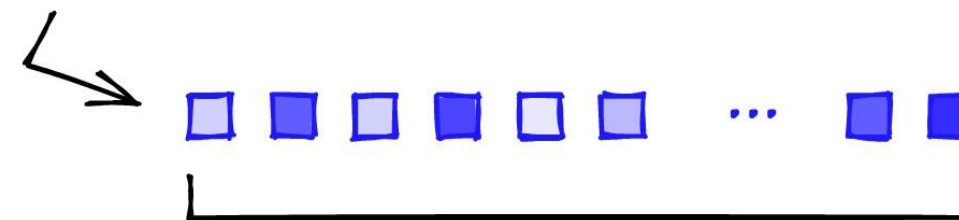
$[0, 0, 0, 1, 0, \dots 0]$



30K+

dense

$[0.2, 0.7, 0.1, 0.8, 0.1, \dots 0.9]$



784

Sparse vs. Dense Vectors

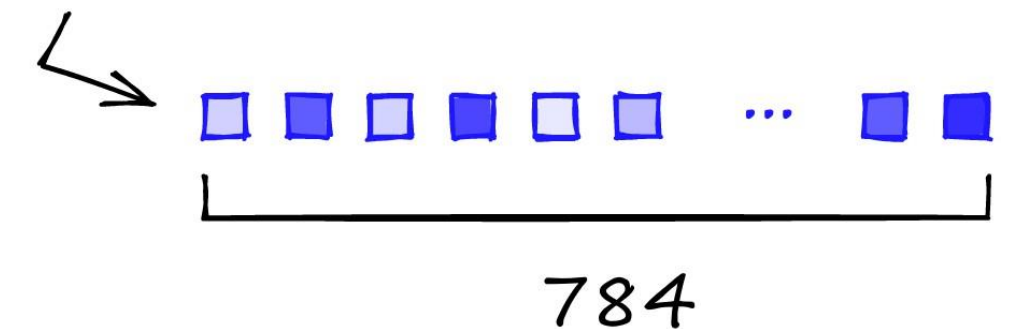
- Why dense vectors?
 - Short vectors may be easier to use as features in machine learning (fewer weights to tune)
 - Dense vectors may generalize better than explicit counts
 - Dense vectors may do better at capturing synonymy:
 - car and automobile are synonyms; but are distinct dimensions in sparse vector
 - a word with car as a neighbor and a word with automobile as a neighbor should be similar, but aren't captured by sparse vector
 - In practice, they work better

How to obtain dense vectors?

- “Neural Language Model”-inspired models
 - Word2vec (skipgram, CBOW), GloVe
- Singular Value Decomposition (SVD)
 - Special case: Latent Semantic Analysis (LSA)
- Alternatives to “static word type embeddings”:
 - Contextual Embeddings (LLM word embeddings)
 - Compute distinct embeddings for a word in its context
 - Separate embeddings for each token of a word

dense

$[0.2, 0.7, 0.1, 0.8, 0.1, \dots 0.9]$



word2vec

word2vec

- Short, dense vector or embedding
- Static embeddings
 - One embedding per word type
 - Does not change with context change
- Two algorithms for computing:
 - Skip-Gram with Negative Sampling or SGNS
 - CBOW or continuous bag of words
 - Popular embedding method
 - Easily available to download and plug in
- Idea: predict rather than count

Mikolov et al., ICLR 2013. Efficient estimation of word representations in vector space.

Mikolov et al., NeurIPS 2013. Distributed representations of words and phrases and their compositionality.

word2vec : Intuition

is traditionally followed by **cherry** pie, a traditional dessert
 often mixed, such as **strawberry** rhubarb pie. Apple pie
 computer peripherals and personal **digital** assistants. These devices usually
 a computer. This includes **information** available on the internet

Instead of counting how often each word w occurs near another, e.g. “cherry”

- Train a classifier on a binary prediction task:
 - Is w likely to show up near “cherry”?

We don't actually care about this task!!!

But we'll take the learned classifier weights as the word embeddings

word2vec: Self-supervision

One missing piece: where to get the (x, y) pairs from?

is traditionally followed by **cherry** pie, a traditional dessert
 often mixed, such as **strawberry** rhubarb pie. Apple pie
 computer peripherals and personal **digital** assistants. These devices usually
 a computer. This includes **information** available on the internet

- A word c that occurs near “cherry” in the corpus acts as the gold “correct answer” for supervised learning
- No need for human labels!

word2vec: predict if candidate word c is a "neighbor"

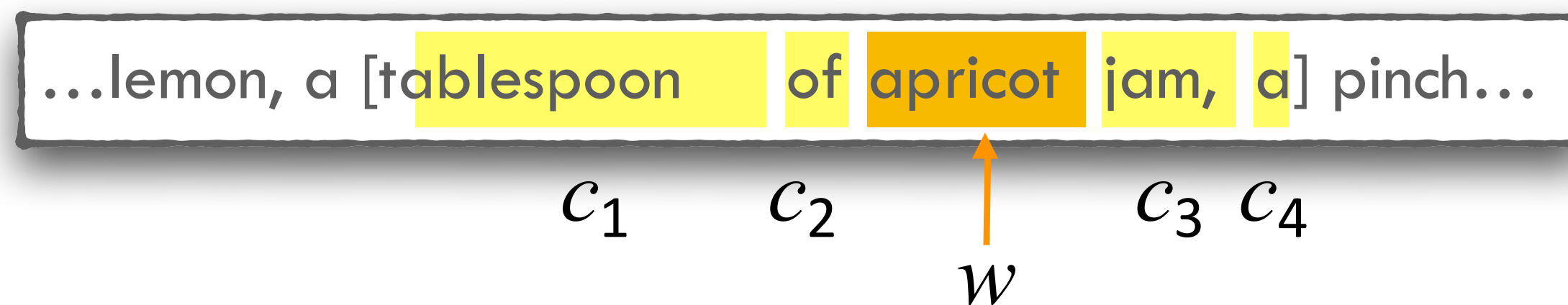
1. Treat the target word t and a neighboring context word c as positive examples.
2. Randomly sample other words in the lexicon to get negative examples
3. Use logistic regression to train a classifier to distinguish those two cases
4. Use the learned weights as the embeddings

word2vec: Goal

Assume a ± 2 word window, given training sentence:

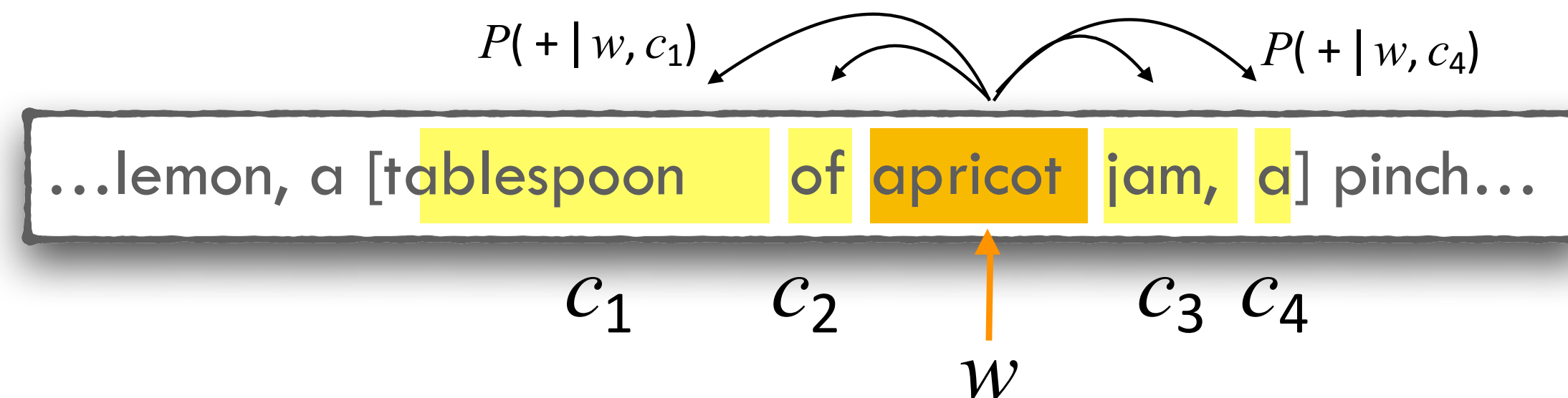
word2vec: Goal

Assume a ± 2 word window, given training sentence:



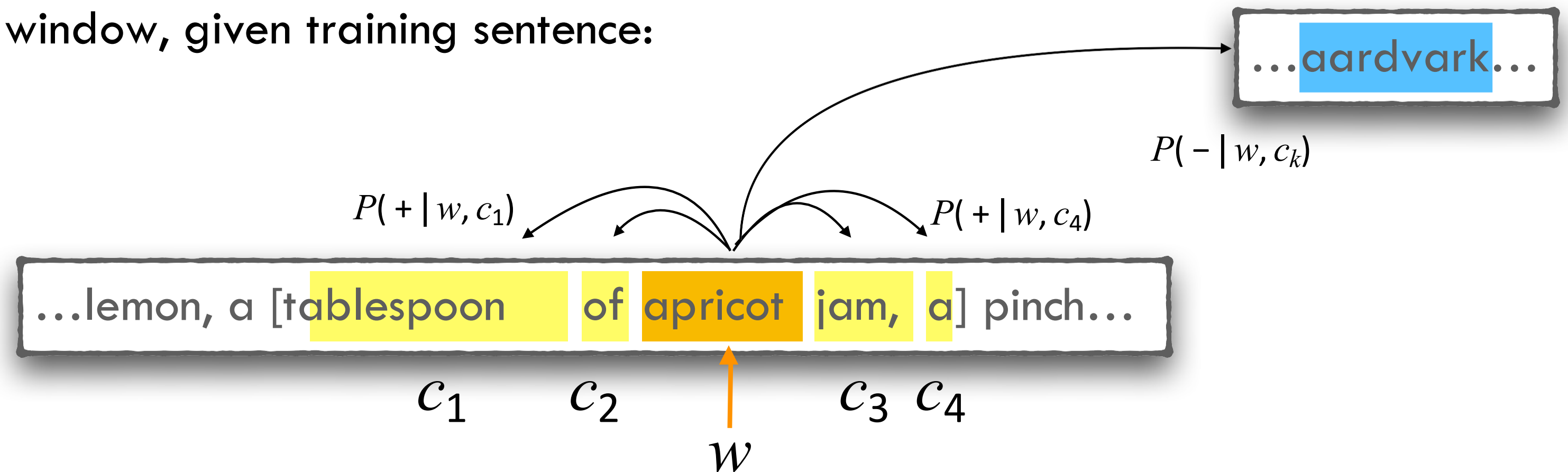
word2vec: Example

Assume a ± 2 word window, given training sentence:



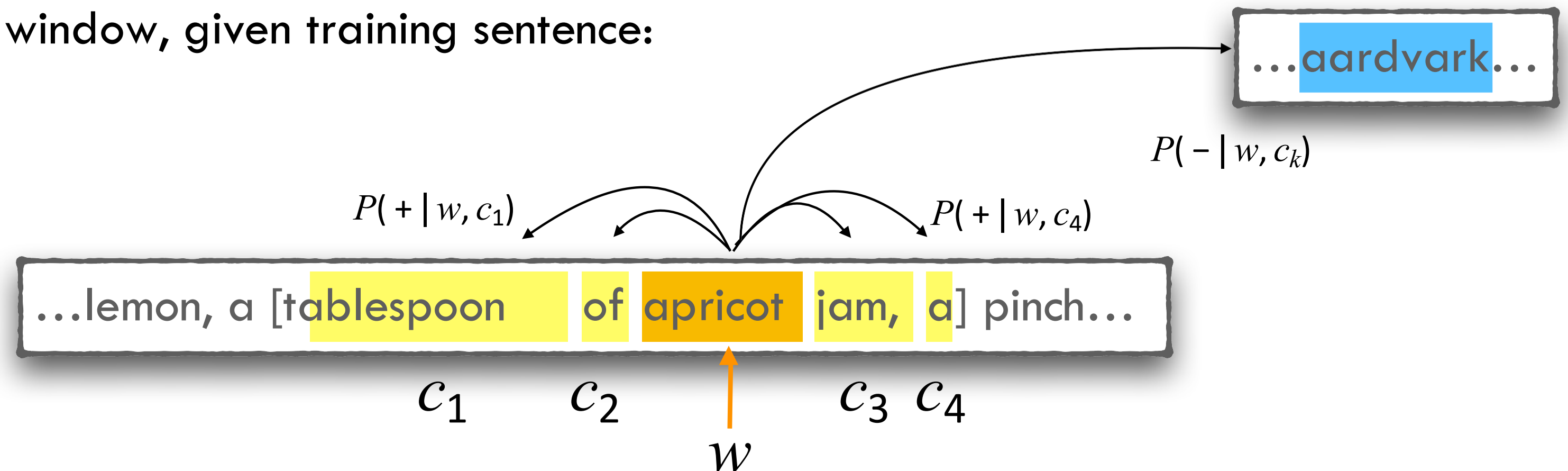
word2vec: Example

Assume a ± 2 word window, given training sentence:



word2vec: Example

Assume a ± 2 word window, given training sentence:



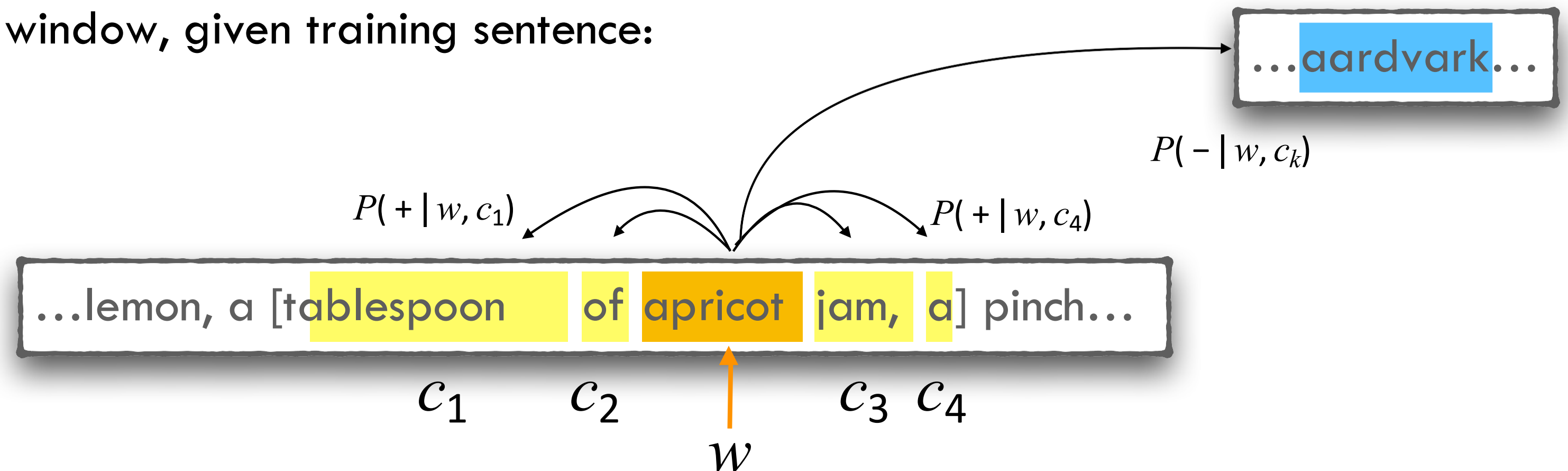
Goal: train a classifier that is given
candidate (word, context) pairs:

(apricot, jam)
(apricot, aardvark)

...

word2vec: Example

Assume a ± 2 word window, given training sentence:



Goal: train a classifier that is given
candidate (word, context) pairs:

(apricot, jam) positive examples
(apricot, aardvark) negative examples
...

And assigns each pair a probability:

$$P(+ | w, c)$$

$$P(- | w, c) = 1 - P(+ | w, c)$$

word2vec: Probability Estimates

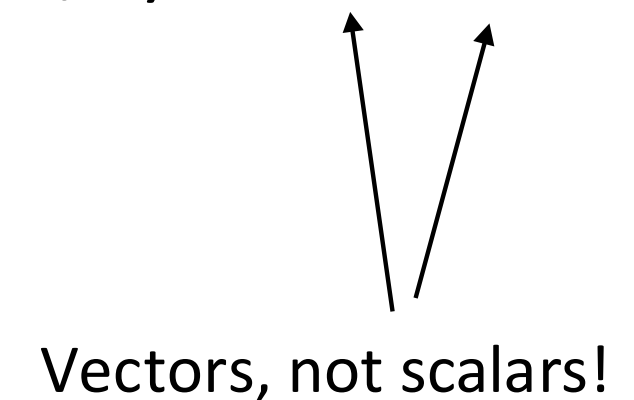
$$P(+ | w, c)$$
$$P(- | w, c) = 1 - P(+ | w, c)$$

- Central intuition: Base this probability on embedding similarity!
- Remember: two vectors are similar if they have a high dot product
 - Cosine similarity is just a normalized dot product

So: Can we just use cosine?

- Cosine still is not a probability!
 - We'll need to normalize to get a probability

$$\text{sim}(w, c) \propto \mathbf{w} \cdot \mathbf{c}$$



Turning dot products into probabilities

Similarity:

$$\text{sim}(\mathbf{w}, \mathbf{c}) \approx \mathbf{w} \cdot \mathbf{c}$$

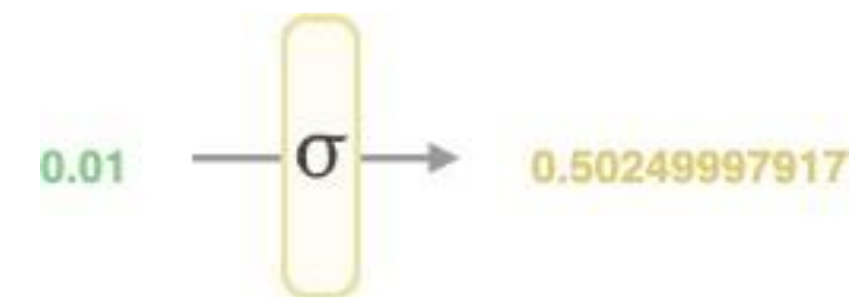
Turn into a probability using the sigmoid function:

$$P(+ | \mathbf{w}, \mathbf{c}) = \sigma(\mathbf{c} \cdot \mathbf{w}) = \frac{1}{1 + \exp(-\mathbf{c} \cdot \mathbf{w})}$$

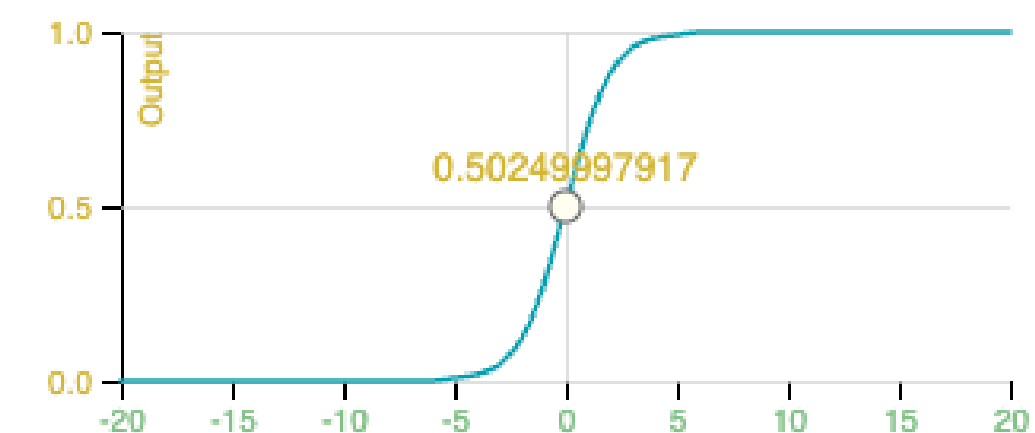
$$P(- | \mathbf{w}, \mathbf{c}) = 1 - P(+ | \mathbf{w}, \mathbf{c})$$

$$= \sigma(-\mathbf{c} \cdot \mathbf{w}) = \frac{1}{1 + \exp(\mathbf{c} \cdot \mathbf{w})}$$

Sigmoid



$$f(0.01) = \frac{1}{1 + e^{-(0.01)}} = 0.50249997917$$

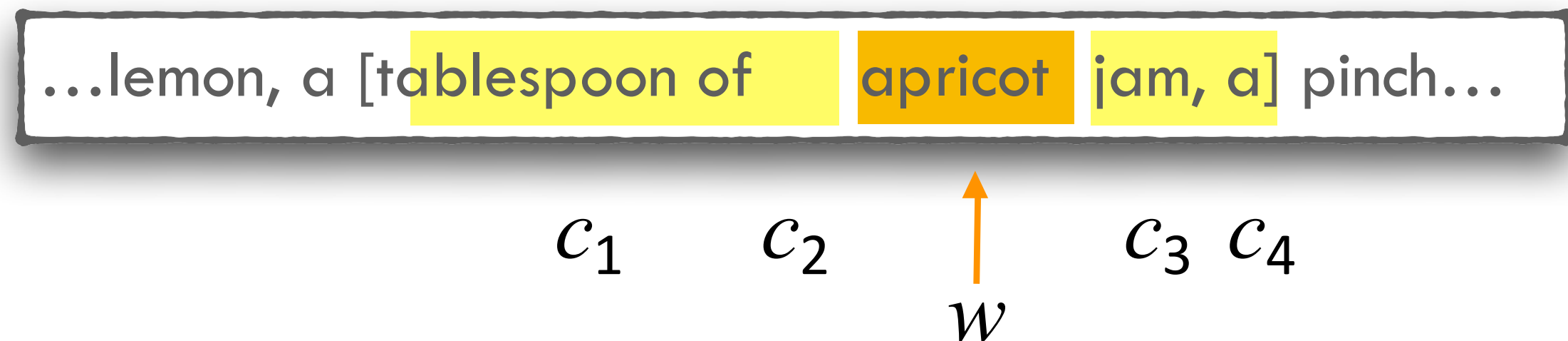


Logistic
Regression!

Accounting for a context window

$$P(+ | w, c) = \sigma(\mathbf{c} \cdot \mathbf{w}) = \frac{1}{1 + \exp(-\mathbf{c} \cdot \mathbf{w})}$$

Single Context Word



But we have lots of context words

- Depends on window size,
- We'll assume independence and just multiply them

Same with negative context words!



$$\log P(- | w, c_{\text{neg}}) = \sum_{c' \in c_{\text{neg}}} \log \sigma(-c' \cdot w)$$

$$P(+ | w, c_{1:L}) = \prod_{i=1}^L \sigma(c_i \cdot w)$$

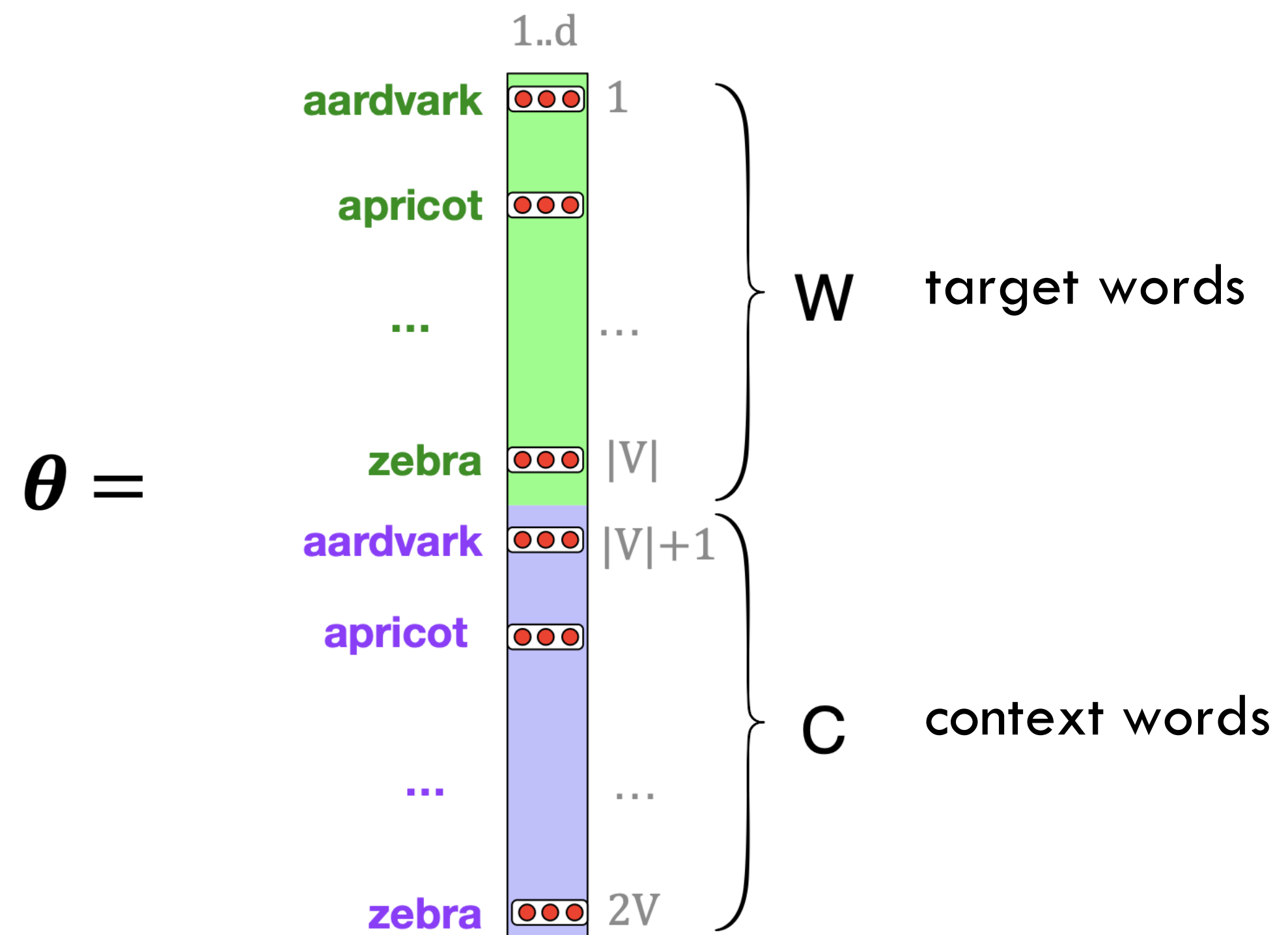
$$\log P(+ | w, c_{1:L}) = \sum_{i=1}^L \log \sigma(c_i \cdot w)$$

word2vec classifier (Skip-gram): Summary

- A probabilistic classifier, given
 - a test target word w
 - its context window of L words $c_{1:L}$
- Estimates probability that w occurs in this window based on similarity of w (embedding) to $c_{1:L}$
- To compute this, we just need embeddings for all the words

word2vec classifier (Skip-gram)

- Separate representations for targets and contexts



Learning word2vec embeddings

Word2vec: Training Data

For each positive example we'll grab a set of negative examples, sampling by weighted unigram frequency

c_{neg}

...aardvark...

...zebra...

...lemon, a [tablespoon of apricot jam, a] pinch...

c_1 c_2 w c_3 c_4

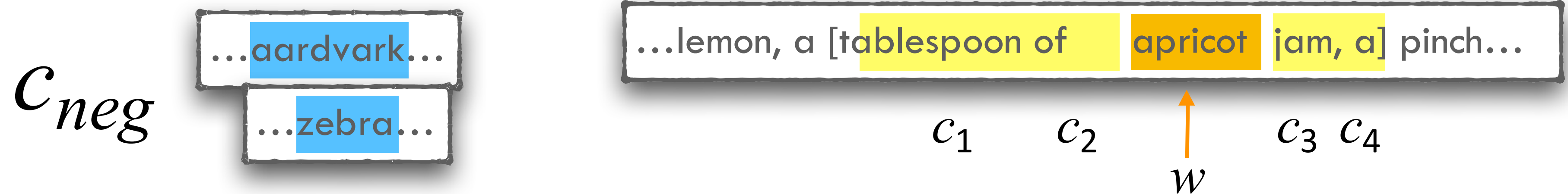
Negative examples

w	c_{neg}
apricot	aardvark
apricot	zebra
apricot	where
apricot	adversarial

Positive examples

w	c
apricot	tablespoon
apricot	of
apricot	jam
apricot	a

Word2vec: Learning Problem



Given

- the set of positive and negative training instances, and
- a set of randomly initialized embedding vectors of size $2|V|$

The goal of learning is to adjust those word vectors such that we:

- Maximize the similarity of the target word, context word pairs $(w, c_{1:L})$ drawn from the positive data
- Minimize the similarity of the (w, c_{neg}) pairs drawn from the negative data

Loss function

Maximize the similarity of the target with the actual context words in a window of size L , and minimize the similarity of the target with the $K > L$ negative sampled non-neighbor words

For every word,
context pair...

$$L_{CE} = -\log \left[P(+|w, c_{pos}) \prod_{i=1}^k P(-|w, c_{neg_i}) \right]$$

Minimize the
negative log
likelihood!

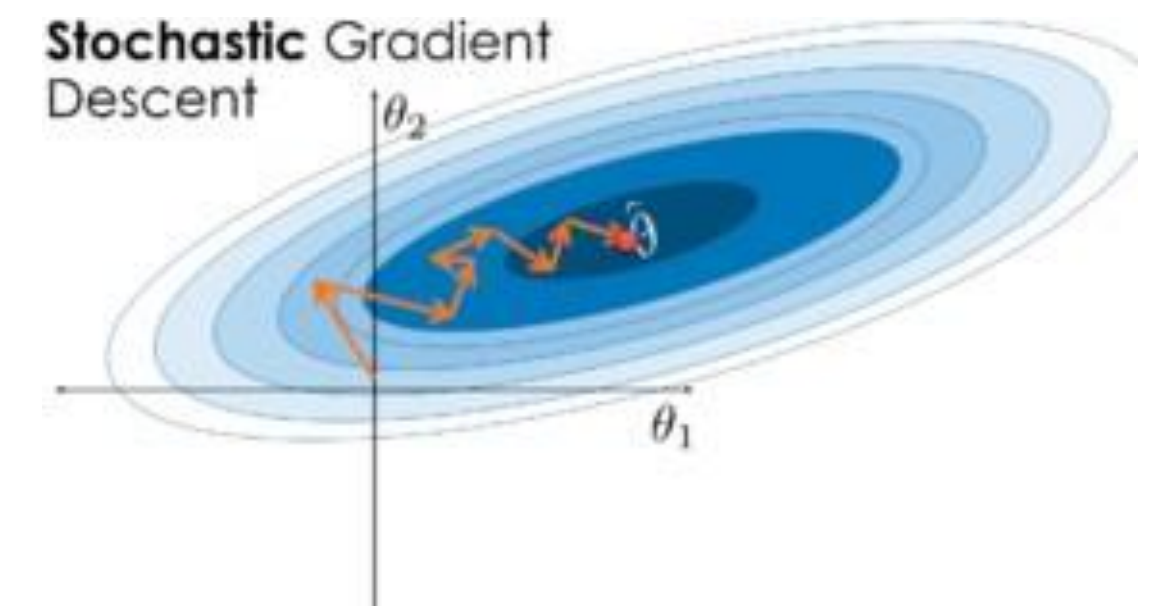
$$= - \left[\log P(+|w, c_{pos}) + \sum_{i=1}^k \log P(-|w, c_{neg_i}) \right]$$

$$= - \left[\log P(+|w, c_{pos}) + \sum_{i=1}^k \log (1 - P(+|w, c_{neg_i})) \right]$$

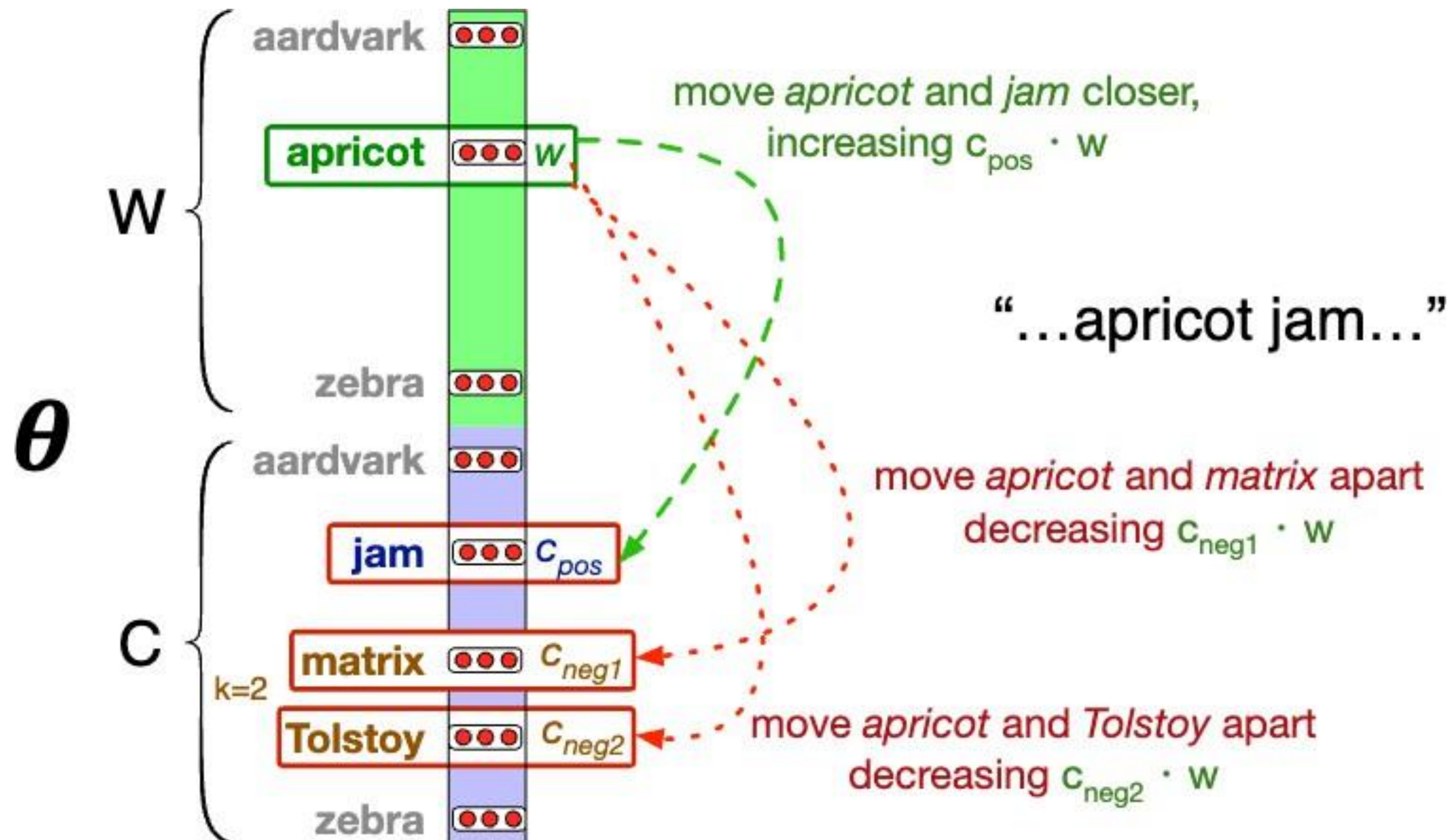
$$= - \left[\log \sigma(c_{pos} \cdot w) + \sum_{i=1}^k \log \sigma(-c_{neg_i} \cdot w) \right]$$

Learning the classifier

- How to learn?
 - Stochastic gradient descent
 - Iterative process
 - Start with randomly initialized weights
 - Update the parameters by computing gradients of the loss w.r.t. parameters
 - Stop when the parameters (or, the loss) do not change much...
- We'll adjust the word weights to
 - make the positive pairs more likely
 - and the negative pairs less likely,
 - over the entire training set.



Intuition of one step of gradient descent



Reminder: gradient descent

At each step:

- Direction: we move in the reverse direction from the gradient of the loss function
- Magnitude: we move the value of this gradient weighted by a learning rate η
- Higher learning rate means move w faster

$$\frac{\partial}{\partial w} L(f(x; w), y)$$

$$w^{t+1} = w^t - \eta \frac{\partial}{\partial w} L(f(x; w), y)$$

SGD: Derivates

$$L_{CE} = - \left[\log \sigma(c_{pos} \cdot w) + \sum_{i=1}^k \log \sigma(-c_{neg_i} \cdot w) \right]$$

3 different parameters

$$\frac{\partial L_{CE}}{\partial c_{pos}} = [\sigma(c_{pos} \cdot w) - 1]w$$

$$\frac{\partial L_{CE}}{\partial c_{neg}} = [\sigma(c_{neg} \cdot w)]w$$

$$\frac{\partial L_{CE}}{\partial w} = [\sigma(c_{pos} \cdot w) - 1]c_{pos} + \sum_{i=1}^k [\sigma(c_{neg_i} \cdot w)]c_{neg_i}$$

SGD: Update

$$c_{pos}^{t+1} = c_{pos}^t - \eta [\sigma(c_{pos}^t \cdot w^t) - 1] w^t$$

$$c_{neg}^{t+1} = c_{neg}^t - \eta [\sigma(c_{neg}^t \cdot w^t)] w^t$$

$$w^{t+1} = w^t - \eta \left[[\sigma(c_{pos} \cdot w^t) - 1] c_{pos} + \sum_{i=1}^k [\sigma(c_{neg_i} \cdot w^t)] c_{neg_i} \right]$$

Two Sets of Embeddings

SGNS learns two sets of embeddings

- Target embeddings matrix W
- Context embeddings matrix C

It's common to just add them together, representing word i as the vector $w_i + c_i$

Summary: word2vec (skip-gram)

Start with $|V|$ random d -dimensional vectors as initial embeddings

Train a classifier based on embedding similarity

- Take a corpus and take pairs of words that co-occur as positive examples
- Take pairs of words that don't co-occur as negative examples
- Train the classifier to distinguish these by slowly adjusting all the embeddings to improve the classifier performance
- Throw away the classifier code and keep the embeddings

Cosine Similarity for Word Similarity

Cosine similarity of two vectors

$$\cos(\vec{v}, \vec{w}) = \frac{\vec{v} \cdot \vec{w}}{|\vec{v}| |\vec{w}|} = \frac{\sum_{i=1}^N v_i w_i}{\sqrt{\sum_{i=1}^N v_i^2} \sqrt{\sum_{i=1}^N w_i^2}}$$

Based on the definition of the dot product
between two vectors $\vec{a}$ and $\vec{b}$

$$\vec{v} \cdot \vec{w} = |\vec{v}| |\vec{w}| \cos \theta$$

$$\cos \theta = \frac{\vec{v} \cdot \vec{w}}{|\vec{v}| |\vec{w}|}$$

Similarity: Cosine Similarity vs. Dot Product

The dot product between two vectors is a scalar:

$$\text{dot product}(\mathbf{v}, \mathbf{w}) = \mathbf{v} \cdot \mathbf{w} = \sum_{i=1}^N v_i w_i = v_1 w_1 + v_2 w_2 + \dots + v_N w_N$$

The dot product tends to be high when the two vectors have large values in the same dimensions:

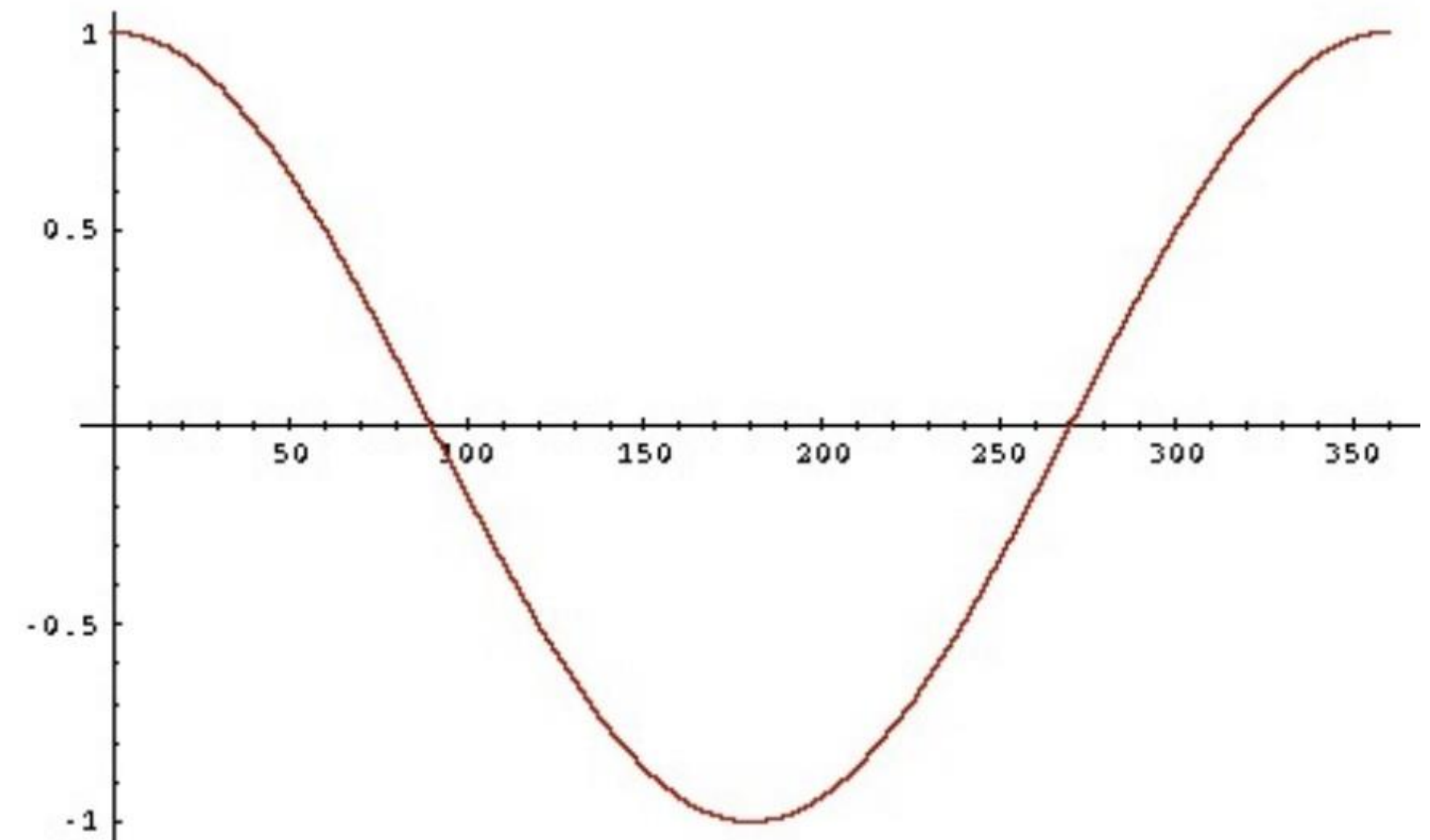
- Dot product favors long vectors
- Dot product is higher if a vector is longer (has higher values in many dimensions)
- Frequent words (e.g., 'the', 'you', 'of') have long vectors (since they occur many times with other words).
- So dot product favors frequent words

Cosine as a similarity metric

-1: vectors point in opposite directions
+1: vectors point in same directions
0: vectors are orthogonal

[normalize out](#)

We do not care about the magnitude of the embeddings, just the angle between them



Cosine Example

$$\cos(\vec{v}, \vec{w}) = \frac{\vec{v} \cdot \vec{w}}{|\vec{v}| |\vec{w}|} = \frac{\vec{v}}{|\vec{v}|} \cdot \frac{\vec{w}}{|\vec{w}|} = \frac{\sum_{i=1}^N v_i w_i}{\sqrt{\sum_{i=1}^N v_i^2} \sqrt{\sum_{i=1}^N w_i^2}}$$

	pie	data	computer
cherry	442	8	2
digital	5	1683	1670
information	5	3982	3325

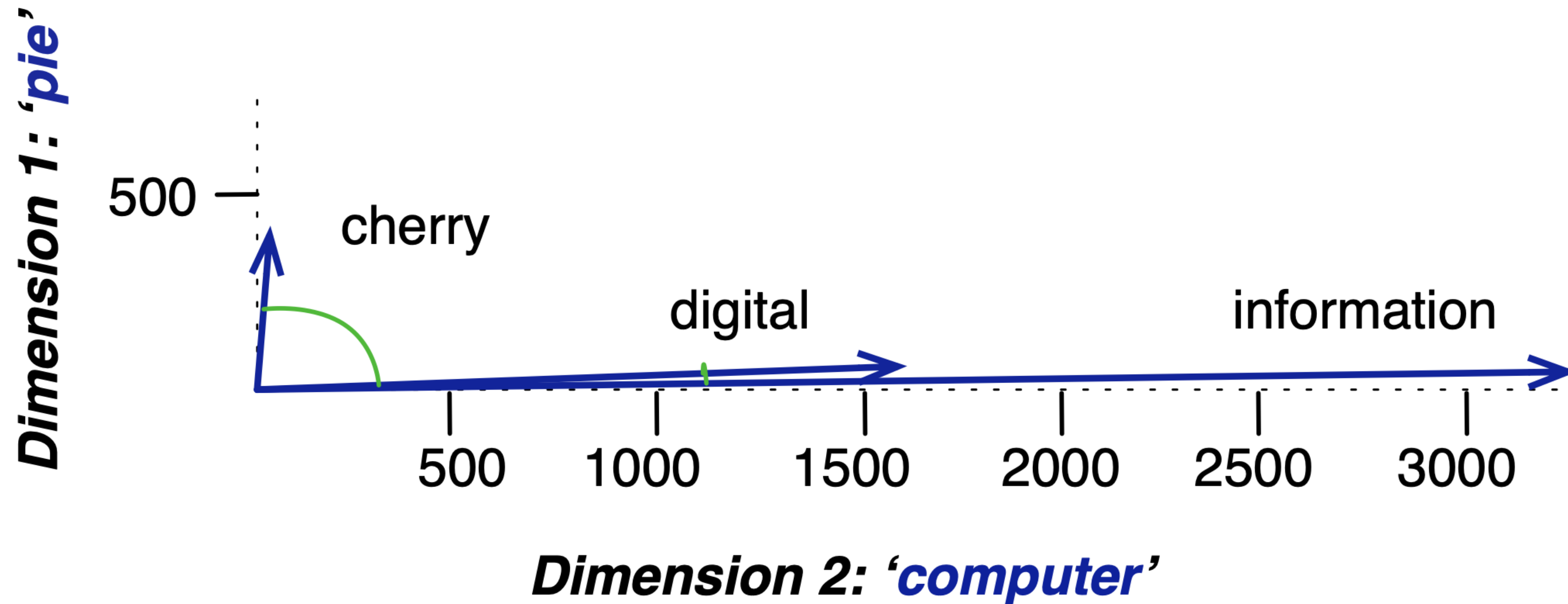
$$\cos(\text{cherry}, \text{information}) =$$

$$\frac{442 * 5 + 8 * 3982 + 2 * 3325}{\sqrt{442^2 + 8^2 + 2^2} \sqrt{5^2 + 3982^2 + 3325^2}} = .017$$

$$\cos(\text{digital}, \text{information}) =$$

$$\frac{5 * 5 + 1683 * 3982 + 1670 * 3325}{\sqrt{5^2 + 1683^2 + 1670^2} \sqrt{5^2 + 3982^2 + 3325^2}} = .996$$

Cosine Example



GloVe

Factorization-based Methods

- Word2Vec is hard to be interpreted
 - Any theoretical insights?

GloVe: Global Vectors for Word Representation

Jeffrey Pennington, Richard Socher, Christopher D. Manning

Computer Science Department, Stanford University, Stanford, CA 94305

`jpennin@stanford.edu, richard@socher.org, manning@stanford.edu`

GloVe

$$\text{PMI}(\text{word}_1, \text{word}_2) = \log_2 \frac{P(\text{word}_1, \text{word}_2)}{P(\text{word}_1)P(\text{word}_2)}$$

Using word vectors to approximate pairwise mutual information (PMI) of two words

GloVe: Global Vectors

- Another very widely used static embedding model
 - model is based on capturing global corpus statistics (not just a narrow window)
 - based on ratios of probabilities from the word-word co-occurrence matrix,
 - intuitions of count-based models like PPMI
- Builds on matrix factorization
 - Idea: store most of the important information in a fixed, small number of dimensions: a dense vector
 - Goal: Create a low-dimensional matrix for the embedding while minimizing reconstruction loss (error in going from low to high dimension)
- Fast training, scalable to huge corpora

GloVe: Matrix-Factorization

- Given all possible target and context words

$$V^T \cdot U \approx \log M$$

- **Solution:** the factorization of M with rank d

Word2Vec vs GloVe

- No particular word embedding approach is the SOTA for all applications
- Some key factors:
 - Amount and quality of training data
 - Hyper-parameters
 - Vector dimension d
 - Negative sampling weight and K
 - Matrix factorization algorithms in GloVe
 -

Apply Word Embeddings to Tasks

Classification

We need a single feature vector to feed into ML classifiers



I



am



a



student



at



USC

Element-wise pooling:

$$v = \text{pool}(v_1, \dots, v_n)$$

e.g. AVG, MAX

No word-order/context information!

Properties and Evaluation of Word Embeddings

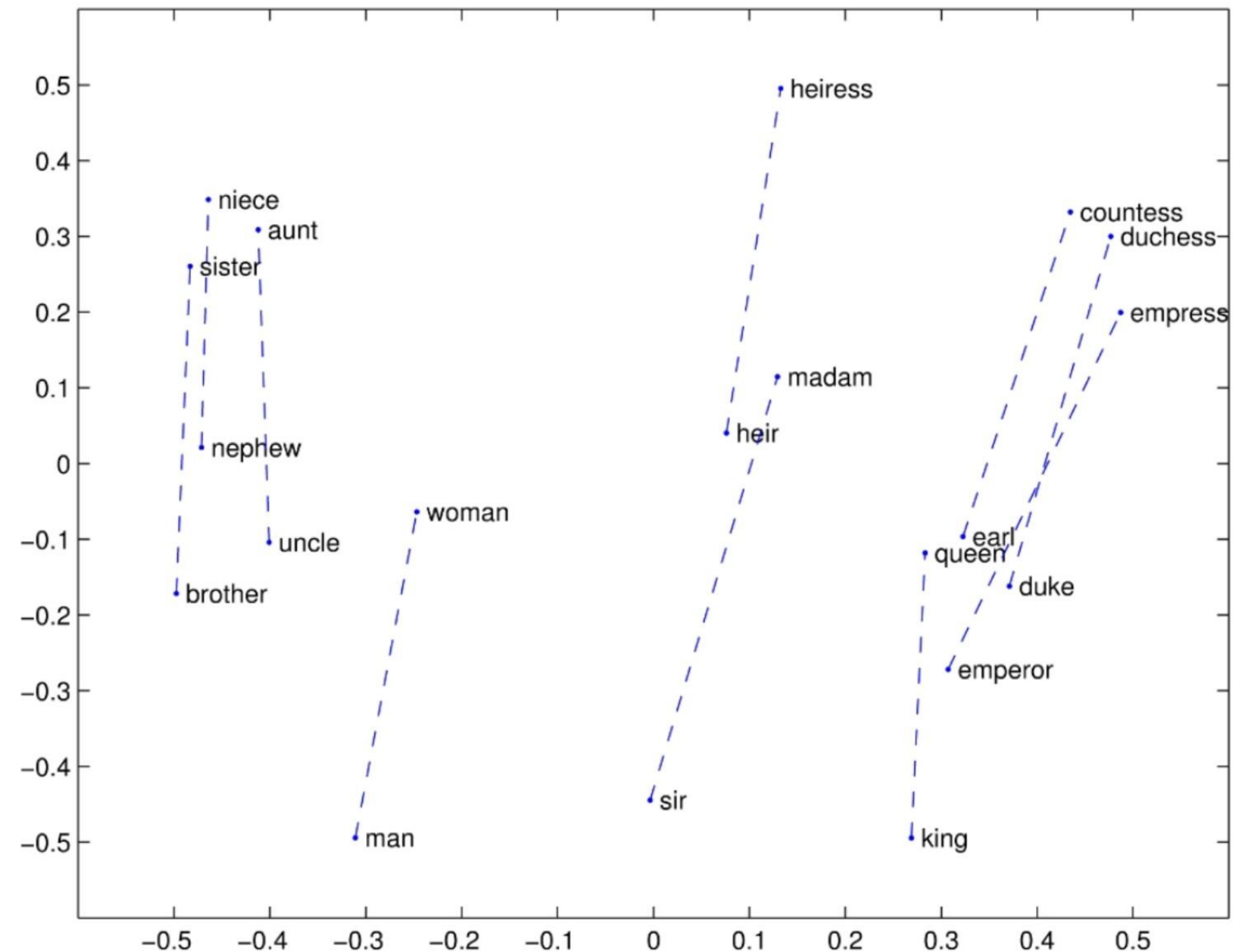
Effects of Context Window Size

Both sparse and dense vectors

- Small windows ($C = \pm 2$) : nearest words are syntactically similar words in same taxonomy (semantics and syntax)
 - Hogwarts nearest neighbors are other fictional schools
 - Sunnydale, Evernight, Blandings
- Large windows ($C = \pm 5$) : nearest words are related words in same topic
 - Hogwarts' nearest neighbors are in the Harry Potter world:
 - Dumbledore, half-blood, Malfoy

Evaluation: via Analogy Relations

- Relational properties of the GloVe vector space, shown by projecting vectors onto two dimensions
 - $\mathbf{W}_{king} - \mathbf{W}_{man} + \mathbf{W}_{woman}$ is similar to $\mathbf{W}_{queen}$
- Caveats: Only works for frequent words, small distances and certain relations (relating countries to capitals, or parts of speech), but not others
 - Understanding analogy is an open area of research
- Better Evaluation: Performance on Downstream Tasks (Extrinsic Evaluation)



Embeddings reflect cultural bias!

Allocational Harms

- Ask “Paris : France :: Tokyo :
Japan”
- Ask “father : doctor :: mother :
nurse”
- Ask “man : computer programmer :: woman :
homemaker”

Algorithms that use embeddings as part of e.g., hiring searches for programmers, might lead to bias in hiring

Embeddings as a tool to study cultural bias!

- Compute a gender or ethnic bias for each adjective: e.g., how much closer the adjective is to "woman" synonyms than "man" synonyms, or names of particular ethnicities
- Embeddings for competence adjective (smart, wise, brilliant, resourceful, thoughtful, logical) are biased toward men, a bias slowly decreasing 1960-1990
- Embeddings for dehumanizing adjectives (barbaric, monstrous, bizarre) were biased toward Asians in the 1930s, bias decreasing over the 20th century
- These match the results of old surveys done in the 1930s

1910	1950	1990
Irresponsible	Disorganized	Inhibited
Envious	Outrageous	Passive
Barbaric	Pompous	Dissolute
Aggressive	Unstable	Haughty
Transparent	Effeminate	Complacent
Monstrous	Unprincipled	Forceful
Hateful	Venomous	Fixed
Cruel	Disobedient	Active
Greedy	Predatory	Sensitive
Bizarre	Boisterous	Hearty

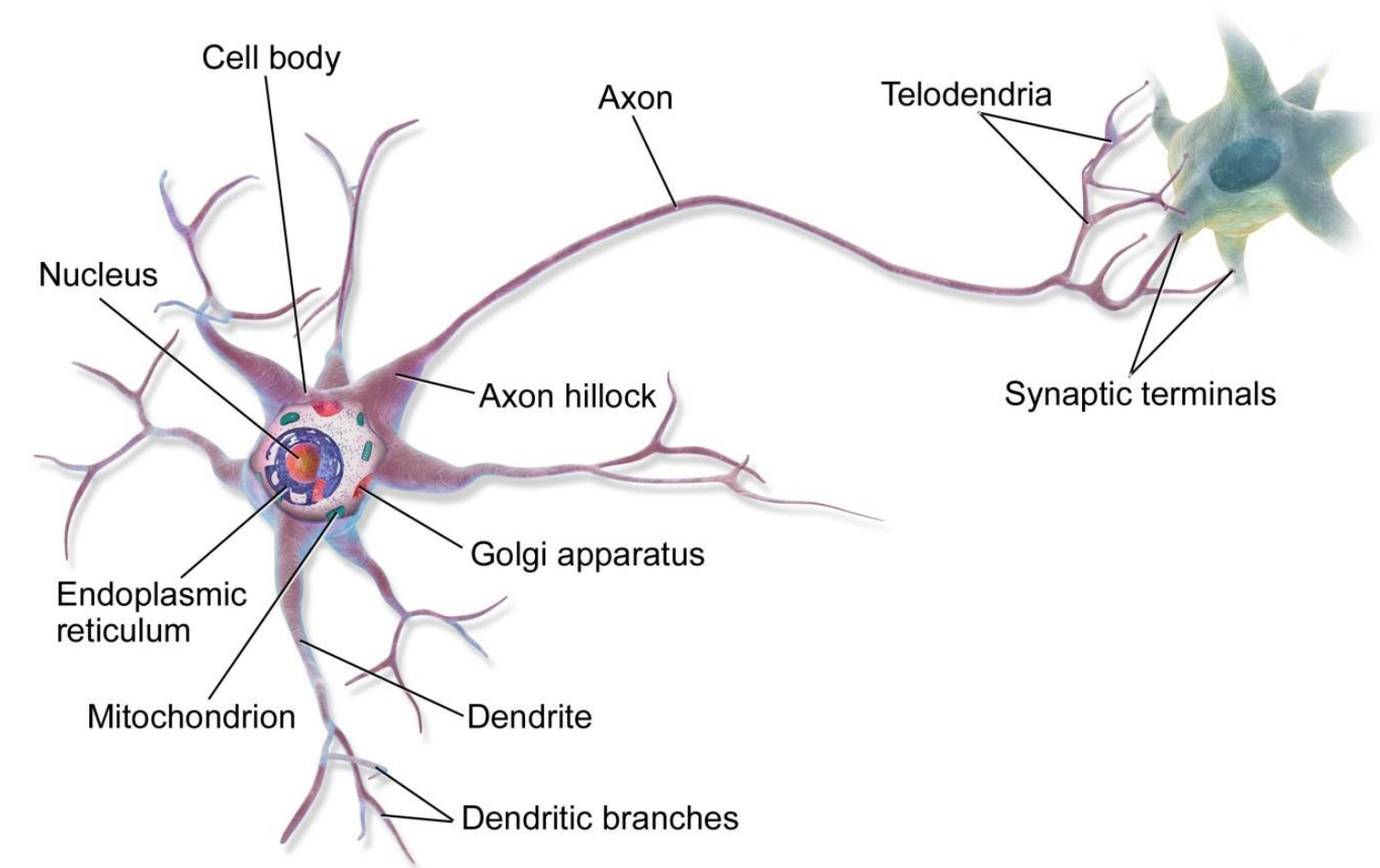
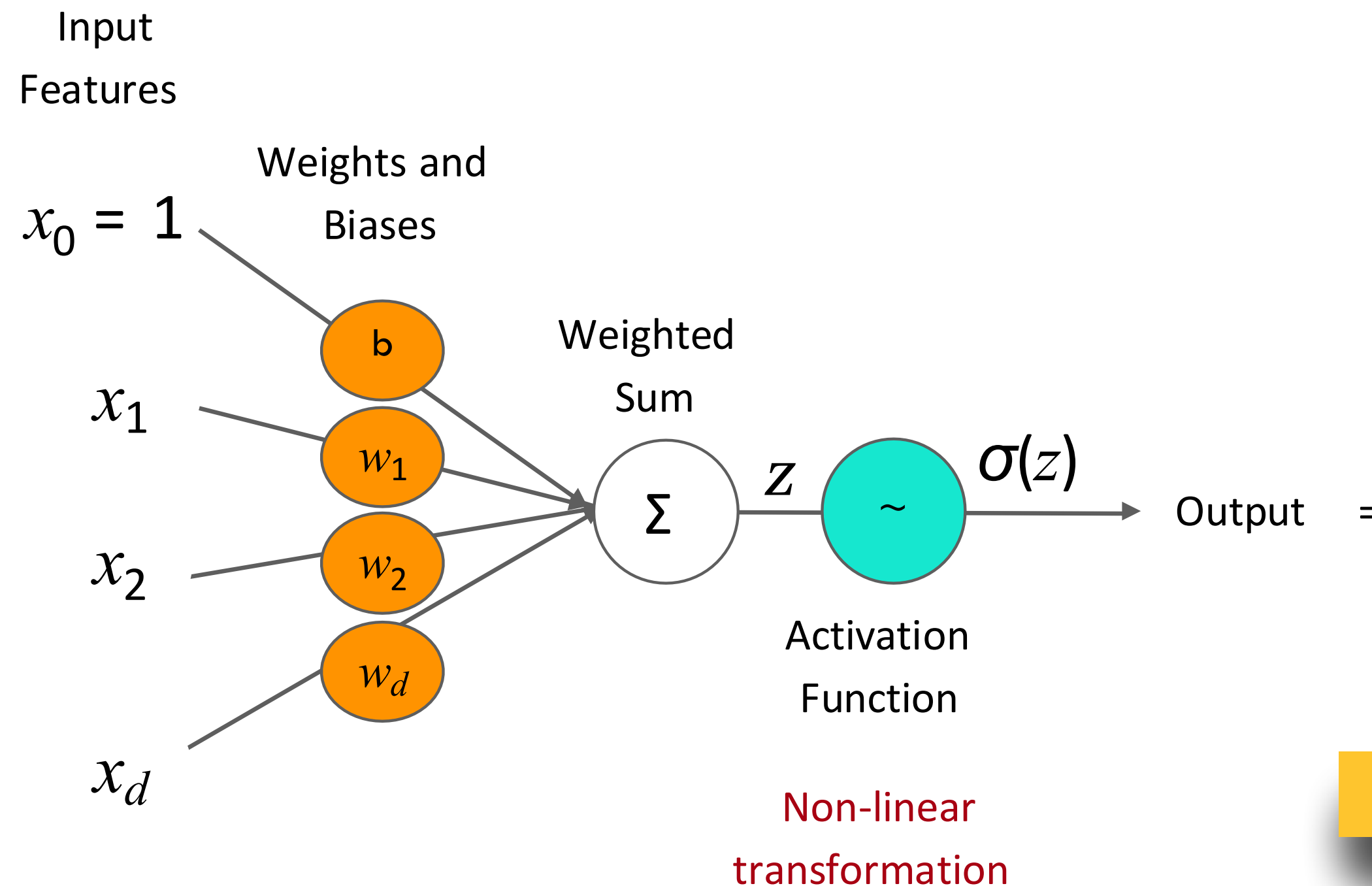
Garg, N., Schiebinger, L., Jurafsky, D., and Zou, J. (2018). Word embeddings quantify 100 years of gender and ethnic stereotypes. *Proceedings of the National Academy of Sciences* 115(16), E3635–E3644.

Representational Harms

Feed-Forward Neural Networks

Neural Network Unit

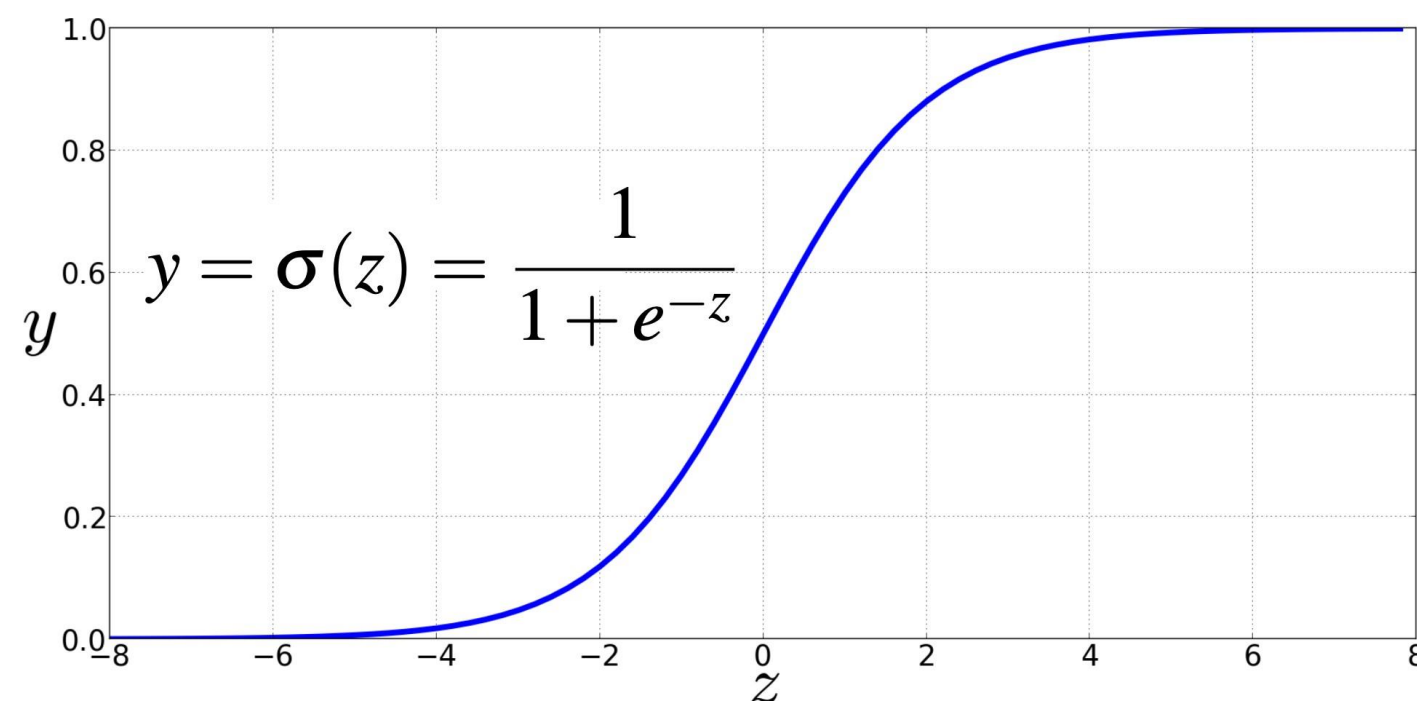
Logistic Regression is a very simple neural network



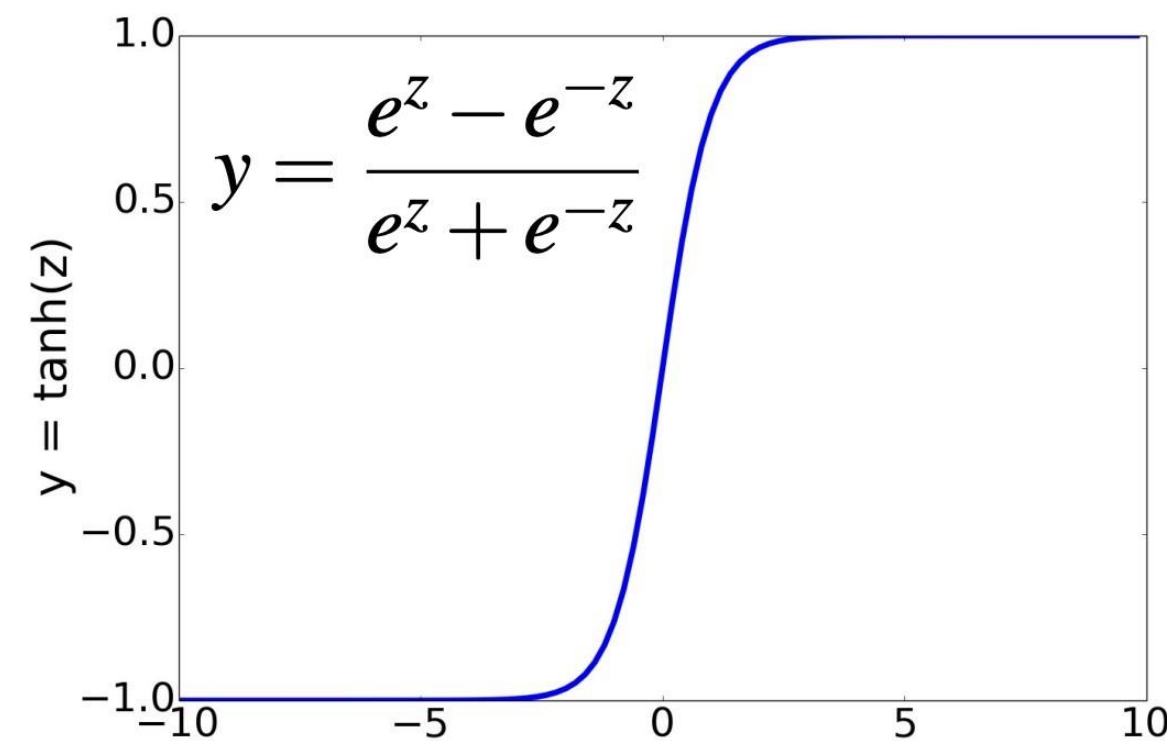
Resembles a neuron in the brain!

Non-Linear Activation Functions

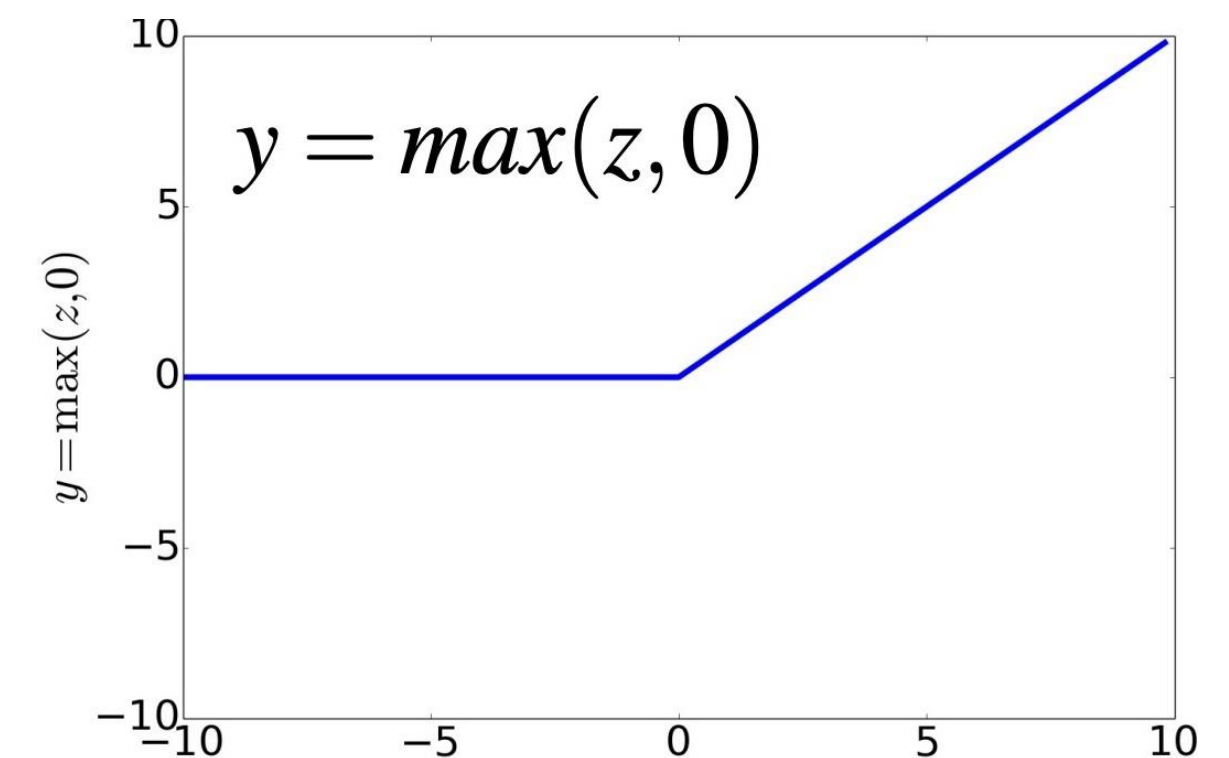
The key ingredient of a neural network is the non-linear activation function



sigmoid



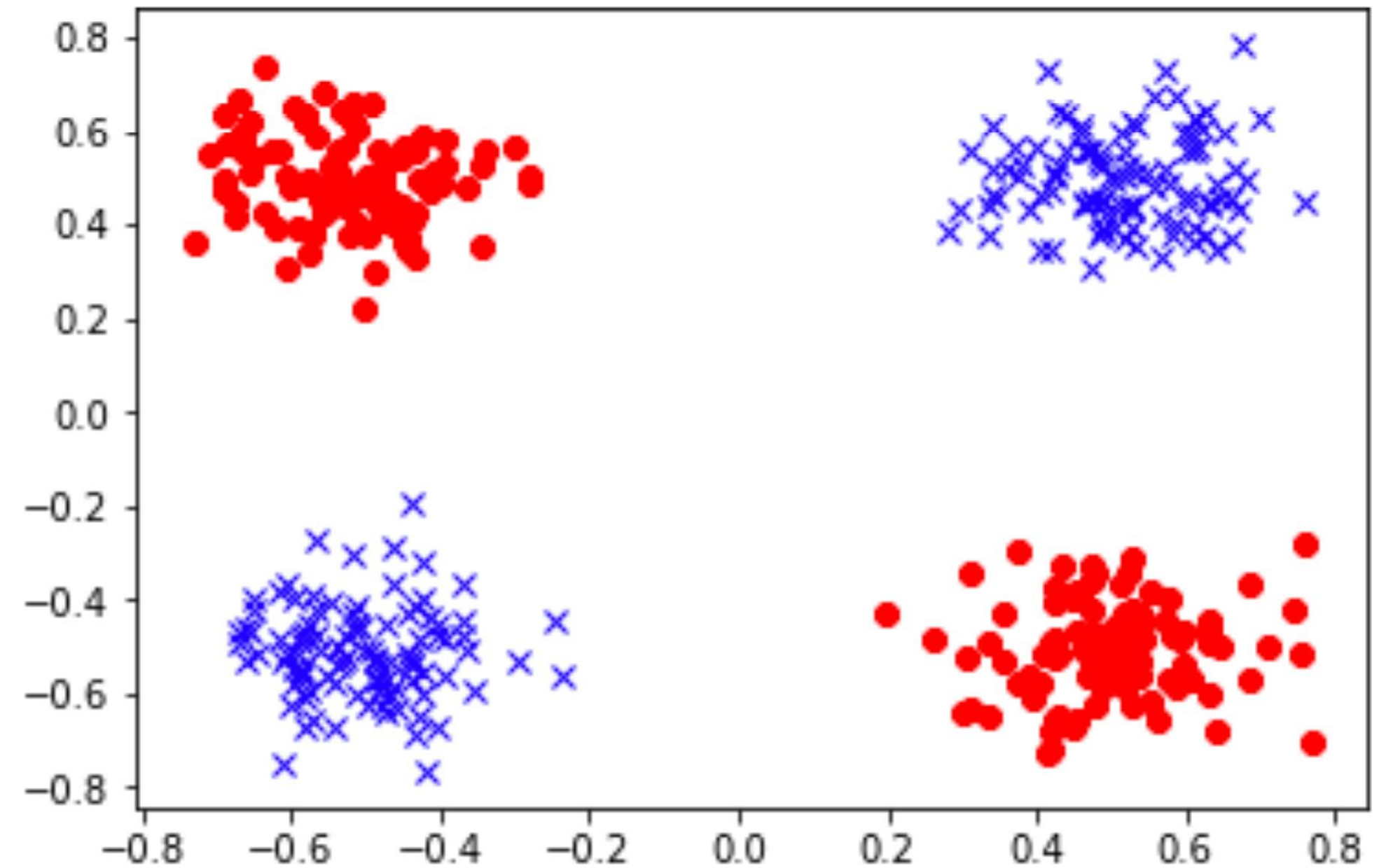
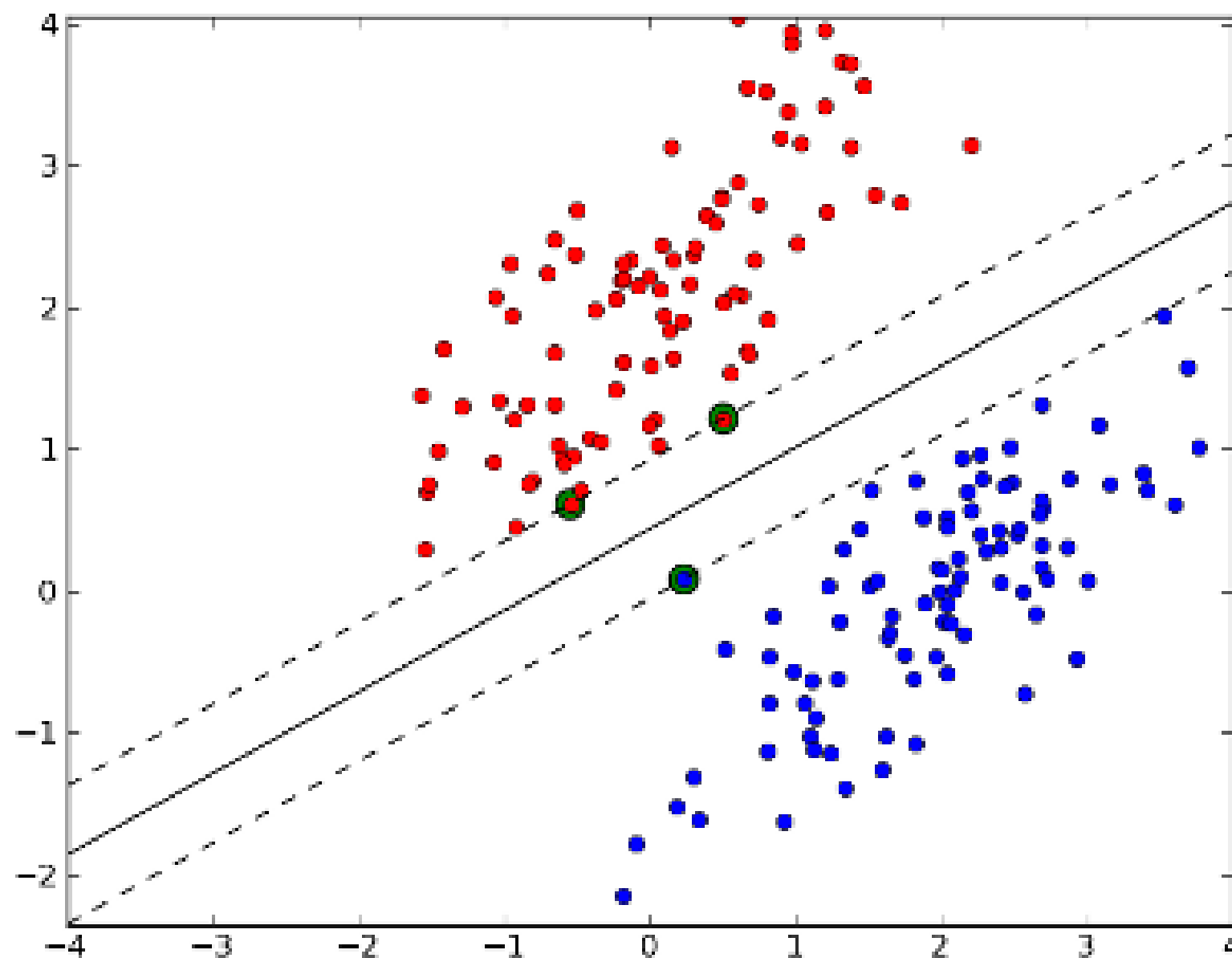
tanh



relu (Rectified Linear Unit)

Most common!

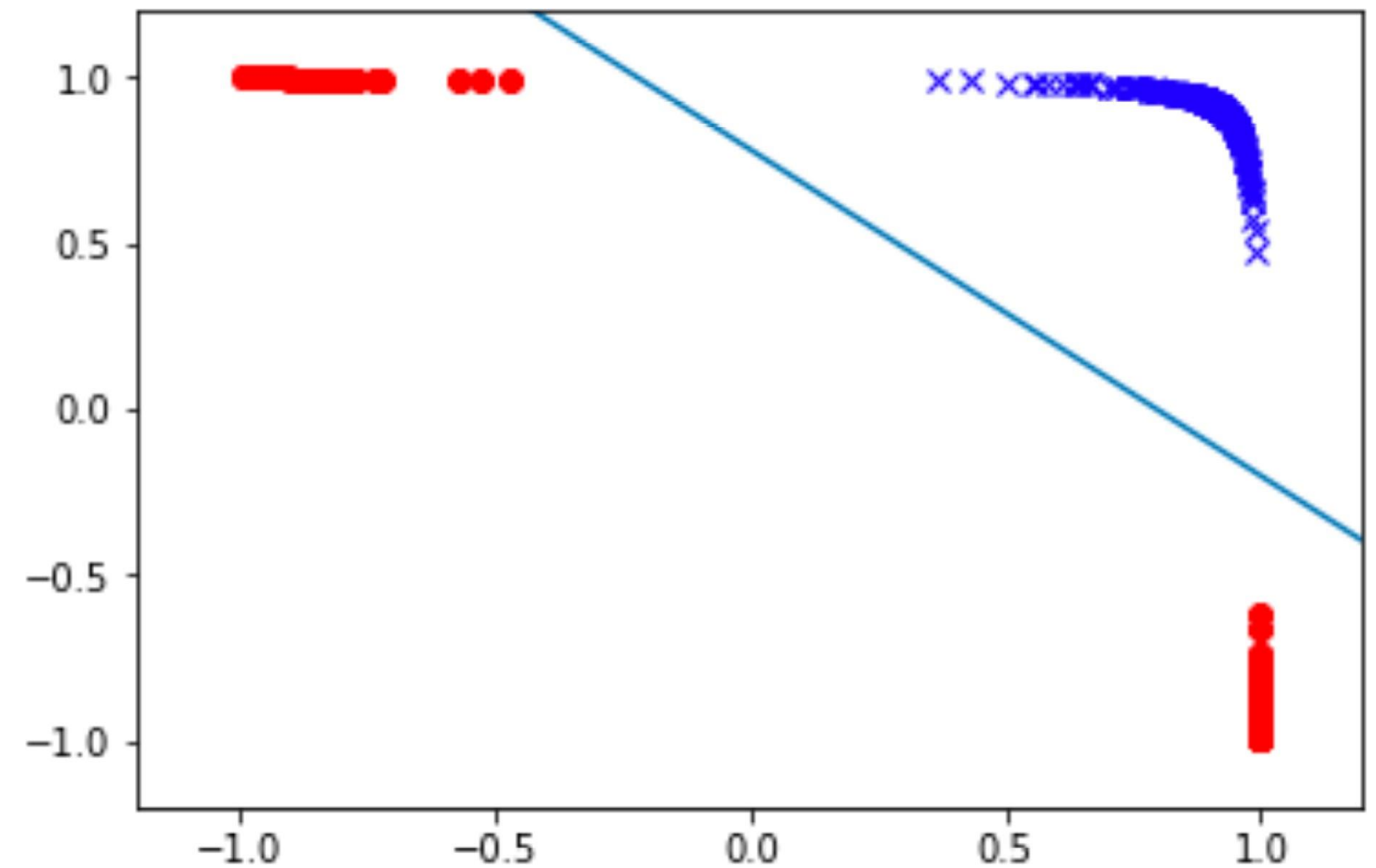
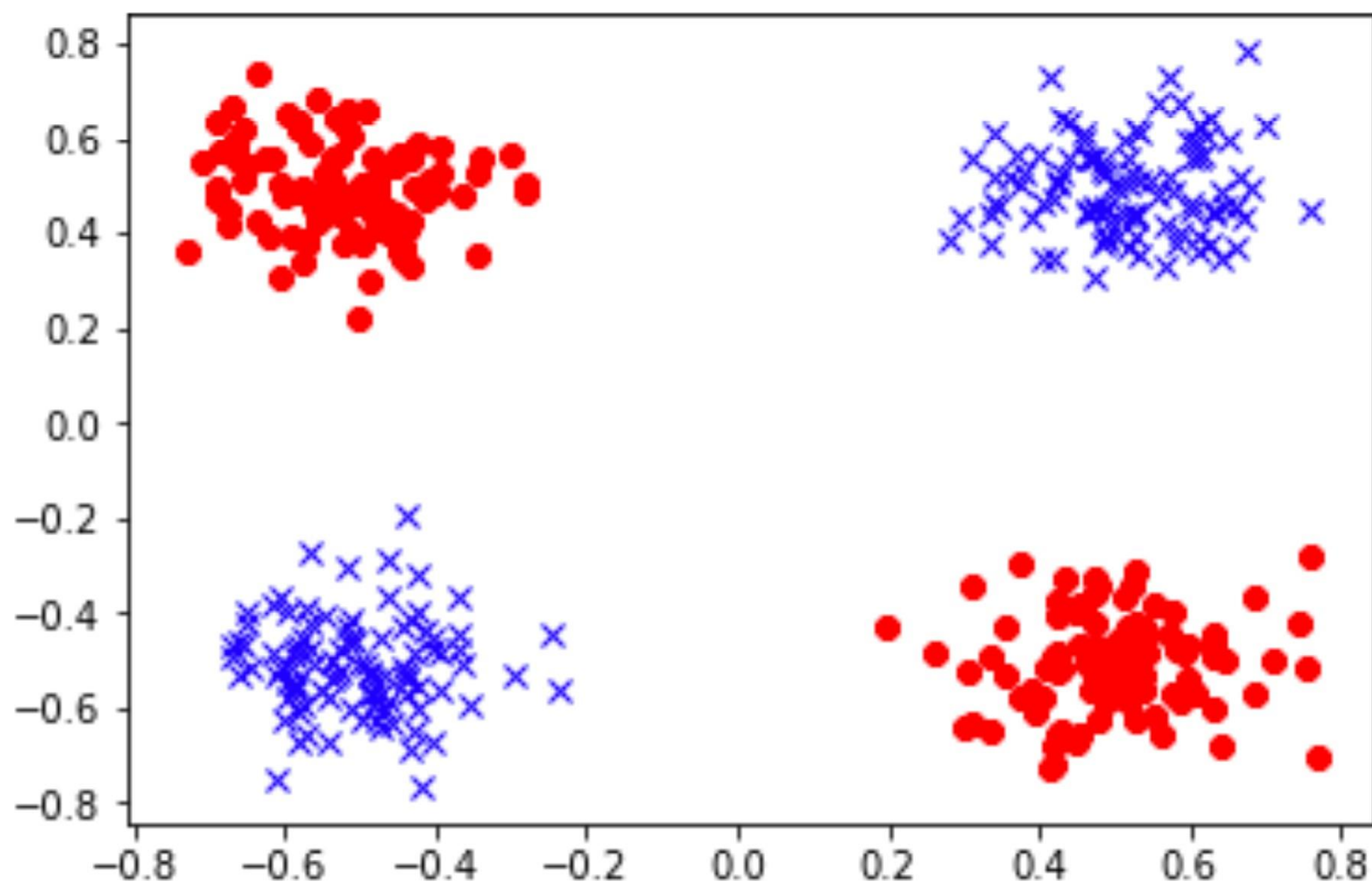
Linear vs. Non-linear Functions



Linearly inseparable

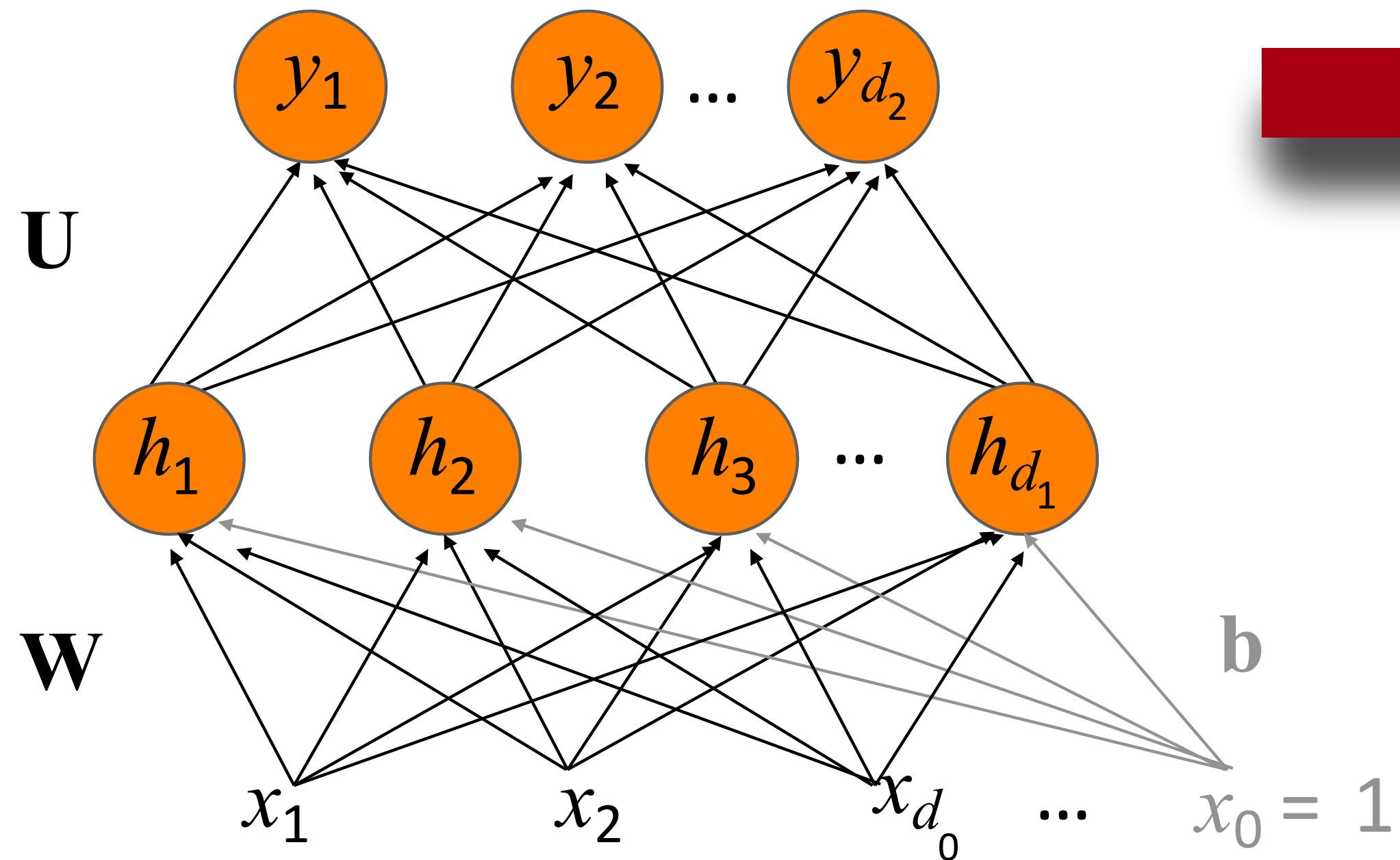
Power of non-linearity

After a $\tanh(\cdot)$ transformation:



Feedforward Neural Nets

Technically, can
learn any function!



Multilayer Perceptron

Let's break it down by revisiting our logistic regression model